

Teaching Exam

Subject – Computer Science

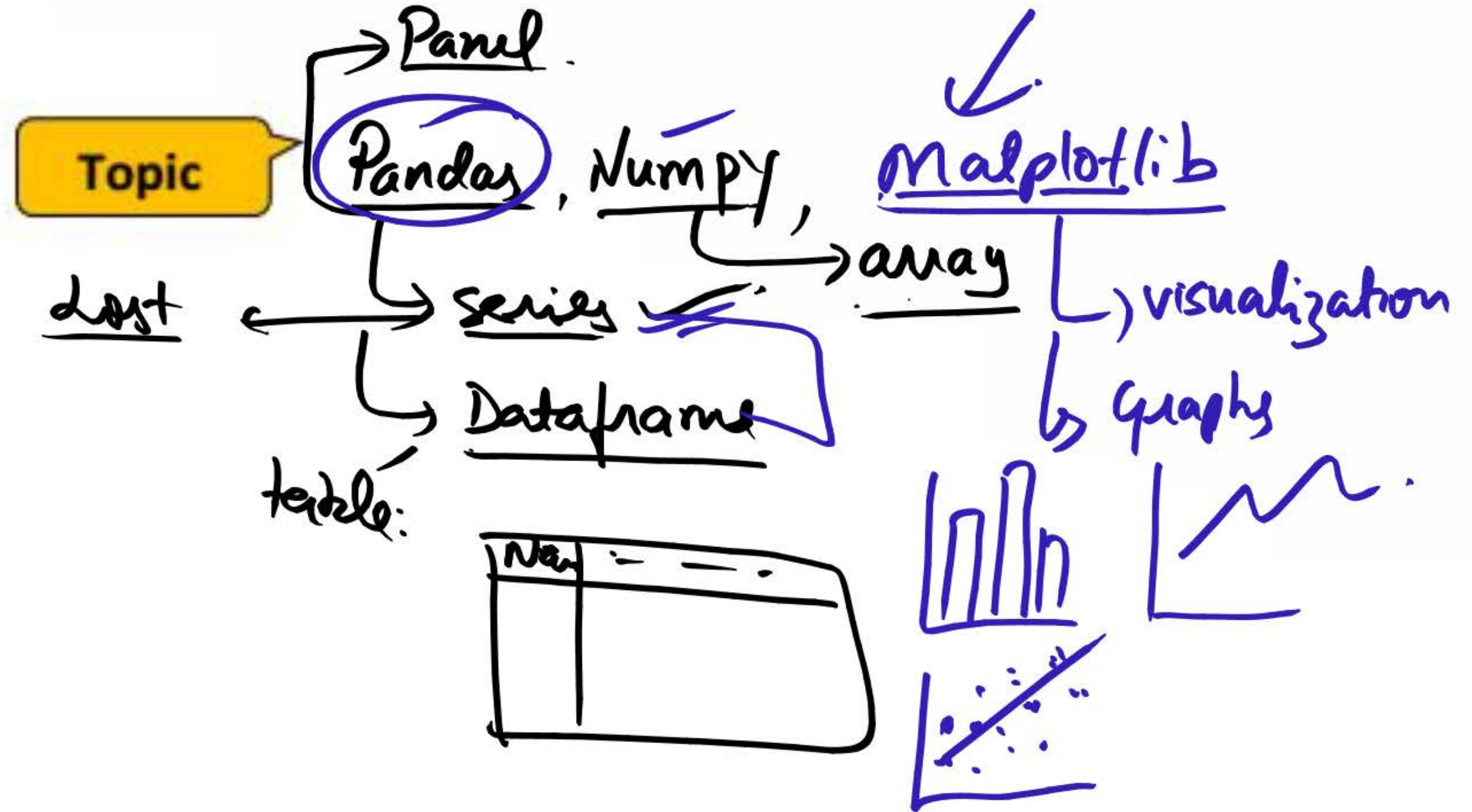
Data Handling Using Pandas



Lecture No –1

By–Satyendra Chaudhary Sir

Topics to be Covered



✅ If you already have Python installed:

▶ Using pip (recommended):

Open your command prompt or terminal and run:

```
bash pip3 install pandas
```

```
pip install pandas
```

To verify it's installed:

Open a Python shell or script and run:

```
python
```

```
import pandas as pd  
print(pd.__version__)
```

If it prints the version (e.g., 2.2.2), you're all set!

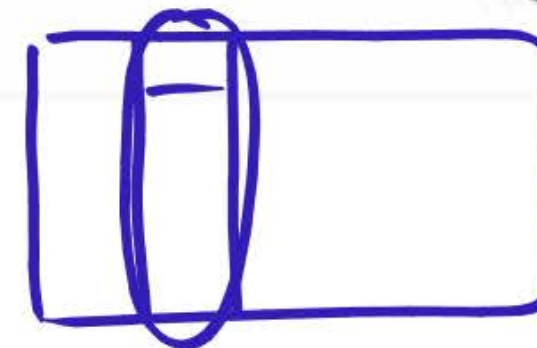
What is Pandas?

Pandas is a powerful Python library used for data analysis, data manipulation, and data cleaning.

Think of it as Excel + SQL + Python, all rolled into one.

It helps you organize, explore, and analyze your data efficiently and quickly.

✓ What Pandas Can Do (Key Features):



Function

What it means

`read_csv()`, `read_excel()`

Load data from files like CSV, Excel, JSON

`DataFrame` and `Series`

→ Data Structure

Store and work with tabular data and columns

Filtering & Sorting

View only the rows you want, or sort them (like SQL `WHERE` or `ORDER BY`)

Handling missing data

Fill, remove, or interpolate missing values easily

Merging & Joining

Combine multiple datasets (like SQL joins)

Grouping & Aggregation

Like Excel's PivotTable or SQL's GROUP BY

Why You Need Pandas:

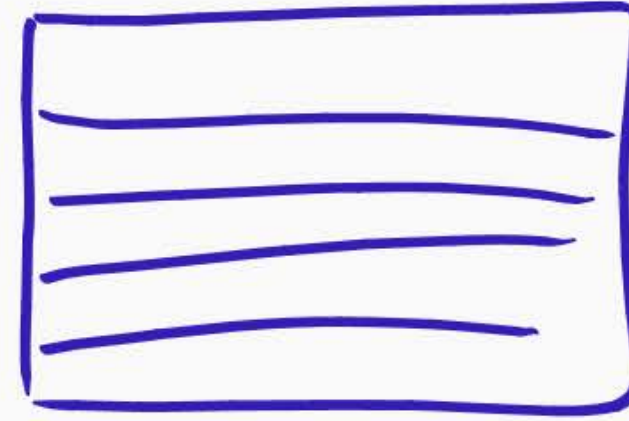
Use Case	How Pandas Helps
<u>Data Analysis</u>	Explore trends, patterns, or anomalies in big datasets
<u>Machine Learning</u>	Clean and prepare data for training ML models
<u>Financial Analysis</u>	Manage stock data, analyze returns, detect outliers
<u>Web Scraping</u>	Organize scraped data into tables and CSVs
<u>Research & Academics</u>	Handle large datasets in education, science, and statistics

1. `read_csv()` – Load Data from CSV

python

```
import pandas as pd
```

```
df = pd.read_csv("students.csv")  
print(df.head())
```



- ◆ This reads a CSV file and loads it into a DataFrame.
- ◆ `df.head()` shows the first 5 rows.

2. DataFrame – Core Table-Like Object

python *import pandas as pd.*
NP

```
data = {  
    'Name': ['Ali', 'Ravi', 'Sara'],  
    'Marks': [85, 90, 78]  
}  
df = pd.DataFrame(data)  
print(df)
```

print(df, index=False)

Series

Output

	Name	Marks
0	Ali	85
1	Ravi	90
2	Sara	78

3. 🔍 Filtering Rows

python

```
high_scores = df[df['Marks'] > 80]  
print(high_scores)
```

- ◆ Shows students with marks greater than 80.

4. 🗝 Handling Missing Values

python

```
df = pd.DataFrame({'Name': ['Ali', 'Sara', 'Ravi'], 'Marks': [85, None, 90]})  
df.fillna(0, inplace=True) # Replace NaN with 0  
print(df)
```

[85, 0, 90]

- Replaces missing values with 0.

Mutable (DataFrame)

5. Merging Two Tables (like SQL JOIN)

python

```
students = pd.DataFrame({'ID': [1, 2], 'Name': ['Ali', 'Ravi']})  
marks = pd.DataFrame({'ID': [1, 2], 'Marks': [85, 90]})  
result = pd.merge(students, marks, on='ID')  
print(result)
```

 Combines two tables using a common column.

6. Grouping & Aggregation

python

```
data = {  
    ✓✓ 'Class': ['A', 'A', 'B', 'B'],  
    ✓✓ 'Marks': [85, 80, 90, 88]  
}  
df = pd.DataFrame(data)  
grouped = df.groupby('Class').mean()  
print(grouped)
```

- ☒ Calculates average marks for each class.

7. 🕒 Time Series Example

python

```
date_rng = pd.date_range(start='2023-01-01', periods=3, freq='D')
df = pd.DataFrame({'Date': date_rng, 'Visitors': [100, 120, 130]})
df.set_index('Date', inplace=True)
print(df)
```

📄 Creates a DataFrame with dates as the index — useful for time series analysis.

Dataframe → [Table], ^{Size} Mutable, Non Homogenous ^{Same}

Series → Column, ^{Size} Immutable, Homogenous, Data values mutable

import pandas as pd.

list1 = ['Raju', 'Prince', 'Gautam']

S = pd.Series(list1)

print(S)

0 Raju
1 Prince
2 Gautam

list
tuple.
(array) → ^{import} numpy as np

import pandas as pd.

list1 = [1, 2, 3, 4, 5]

→ S = pd.Series(list1, index=["A", "B", "C", "D", "E"])

False:

A 1
B 2
C 3
D 4
E 5

Series attributes

Series.index ✓

Series.size ✓

Series.name ✓

Series.dtype ✓

Series.nbytes ✓

Series.ndim ✓

Series.itemsize ✓

Series.empty ✓

Series.shape ✓

[5, 8, 9.5]

float

import pandas as pd.

`S = pd.Series([1, 2, 3, 4, 5], index=["A", "B", "C", "D", "E"], name='Name')`

`print(S.size)` ~~##~~ Output 5.

`print(S.name)` ~~##~~ Name

`print(S.empty)` ~~##~~ False



2 mins Summary



TEACHING
WALLAH

Topic

Data Frame

File reader

Operation

THANK YOU